Distributed Multi-GPU Sparse Matrix-Vector Multiplication (SpMV) via 1D Cyclic Partitioning and GPU-Aware Communication

Michael Bernasconi
Student ID: 267681
michael.bernasconi@studenti.unitn.it
https://github.com/Michael-Bernasconi/Sparse-Matrix-Vector-Multiplication/tree/Deliverable2
Department of Information Engineering and Computer Science
University of Trento

***Abstract*—Distributed Sparse Matrix-Vector multiplication (SpMV) represents a vital bottleneck in massive-scale numerical computing, where efficiency is heavily constrained by structural irregularity, network latency, and load imbalance. This study extends single-node architectures to a multi-GPU environment by evolving a continuous block partition layout into a 1D cyclic distribution ($owner(i) = i \pmod P$). By completely removing high-overhead global collective broadcast procedures (MPI_Bcast), a custom point-to-point sparse communication overlay is implemented. This layout tracks, isolates, and exchanges exclusively the non-local active components of the multiplier vector x (Ghost Entries) through non-blocking peer-to-peer routines. Furthermore, intermediate host-staged buffer paths are eliminated by refactoring the pipeline with GPU-Aware MPI paradigms operating directly on device memories. Performance evaluations conducted on the University of Trento cluster (Ampere A30 GPUs) demonstrate that while 1D cyclic mapping perfectly balances uniform data distributions (e.g., ASIC structures), highly skewed power-law structures remain strictly bound by communication overhead, providing clear experimental insights regarding the intersection of data topology, interconnect bandwidth, and multi-GPU scalability.**

***Index Terms*—SpMV, Multi-GPU CUDA, 1D Cyclic Partitioning, Ghost Entries Exchange, GPU-Aware MPI, Load Balancing, Performance Interconnect Bound, Strong Scaling, Weak Scaling.**

I. Introduction

Sparse Matrix-Vector multiplication ($y = A \times x$) is the underlying backbone of modern scientific applications, differential equation solvers, and graph analytics. Evolving single-node GPU kernels to distributed architectures introduces severe scalability roadblocks caused by non-contiguous memory layouts and network bottlenecks.

While the initial project implementation focused on a standard 1D continuous block-row slicing scheme, real-world matrices from the SuiteSparse collection exhibit non-uniform sparsity patterns that result in major computational idle times (load imbalance) and excessive memory overhead due to redundant vector replication.

This deliverable implements a 1D cyclic row-distribution framework designed to optimize parallel workload mapping across multiple hardware devices. By rewriting the communication infrastructure around localized peer-to-peer sparse exchanges, the system eliminates collective broadcast overheads. This report details the core design principles, structural transformations, and benchmarking outcomes of the distributed SpMV execution campaign across strong and weak scaling configurations.

II. Parallel Design Methodology

A. Evolution via Foster's Design Strategy

To effectively transition single-device execution paths into an enterprise-grade multi-GPU paradigm, the application architecture was evolved adhering strictly to Foster's four-stage design methodology:

- **Partitioning:** Fine-grained domain decomposition is performed on the row primitives of matrix A. In accordance with the required baseline configuration, Rank 0 sequentially reads the entire Matrix Market file from disk and handles the distribution of matrix entries to all other processes. The assignment of global row indices to target cluster ranks follows a precise modular distribution scheme, formally defined as:

$$owner(i) = i \pmod{\text{size}} \tag{1}$$

  where i represents the global row index and size corresponds to the number of active MPI ranks (GPUs).
- **Communication:** Rather than distributing the entire multiplier vector x via global collectives, a specialized discovery process identifies non-local indices requested by local non-zero elements (col_idx). These indices represent the matrix *Ghost Entries*, which are captured in isolated communication descriptors.
- **Agglomeration:** Rows assigned to a single executor are agglomerated into an independent, compact local sparse matrix representation (local CSR or COO layouts), converting the sparse structure into relative coordinate systems to ensure zero index-translation overhead during raw kernel execution.
- **Mapping:** Each MPI process is pinned to a dedicated hardware device ($NVIDIA\ A30\ GPU$), executing localized data processing routines natively within isolated CUDA streams.

B. GPU-Aware Point-to-Point Communication Overlay

To fulfill strict efficiency and high-throughput requirements, intermediate host-side staging operations are entirely bypassed. Standard multi-GPU implementations pull device results to host memory buffers via high-latency `cudaMemcpy` transactions prior to invoking MPI routines.

This architecture leverages a **GPU-Aware MPI** runtime. Because the underlying network stack safely decodes CUDA virtual memory spaces, device pointers are supplied directly to non-blocking point-to-point primitives (`MPI_Irecv` and `MPI_Isend`), followed by an explicit `MPI_Waitall` barrier. The trade-offs between redundant numerical computations and communication framework penalties have been widely investigated within dense multi-GPU clusters to elevate iterative execution behaviors [2].

Algorithm 1: Distributed GPU-Aware SpMV Pipeline

Input: Local Sparse Matrix (A_{loc}), Managed Vector Segment (x_{loc})
Output: Fully Synchronized Interleaved Output Vector (y_{global})
Scan col_idx to populate Ghost Receive and Send Descriptors
Allocate Device Buffers `d_ghost_recv` and `d_ghost_send`

```
// 1. Pack local values requested by others
pack_ghost_kernel<<<(...)>>>(...)
// 2. GPU-Aware Peer-to-Peer Non-blocking
   Exchange
MPI_Irecv(d_ghost_recv, ..., src_rank, ...,
 &reqs[0])
MPI_Isend(d_ghost_send, ..., dest_rank, ...,
 &reqs[1])
MPI_Waitall(2, reqs, MPI_STATUSES_IGNORE)
```

Start `cudaEvent_t` Computational Timers
Launch Selected CUDA SpMV Kernel (CSR, COO, or Vector)
Stop `cudaEvent_t` Computational Timers

```
MPI_Gatherv(d_local_y, ..., d_interleaved_y,
 ..., Root)
```

if *Rank == 0* **then**
  De-interleave `d_interleaved_y` into linear vector y_{global}
end

III. Experimental Setup & Dataset

A. Hardware Infrastructure

All execution cycles were evaluated on the `edu01` compute node (`edu-short` partition) of the University of Trento cluster. The exact hardware specifications are detailed in Table I. All configurations utilized CUDA v12.3.2 and OpenMPI v4.1.5 with native CUDA-Aware support enabled.

TABLE I: Target Hardware Specifications (UniTN Cluster)

Component	Host (CPU)	Device (GPU)
Model	Intel(R) Xeon(R) Silver 4309Y	NVIDIA A30
Architecture	Ice Lake (x86_64)	Ampere (Compute Cap. 8.0)
Cores / SMs	16 Cores / 32 Threads (2 Sockets)	56 SMs (3584 CUDA Cores)
Frequency	2.80 GHz (Base)	1.44 GHz (Boost)
FP32 Perf.	1.84 TFLOPS	10.3 TFLOPS
Memory BW	102.4 GB/s	933.1 GB/s (HBM2)
Global Mem.	N/A	24 GB
L2 Cache	20 MiB	24 MiB

B. Dataset Optimization Strategy

To guarantee a robust performance profile covering both Strong and Weak Scaling benchmarks, the experimental matrix framework is divided into two categories: real-world structures for Strong Scaling, and algorithmically generated structures for Weak Scaling.

Compared to the previous single-GPU experimental campaign, the target selection of sparse datasets has been intentionally modified and optimized. This adaptation was required since several larger matrices tested in the single-node setup presented structural scaling blockages and introduced high memory allocation problems or outright execution failures when distributed across the current multi-GPU cluster overlay. Thus, the workload selection has been revised to preserve evaluation stability while properly maintaining topological variance and uneven sparsity domains.

1) Strong Scaling Dataset: The baseline dataset comprises six distinct real-world sparse structures from the SuiteSparse Matrix Collection, carefully selected to evaluate the algorithms against different structural domains.

TABLE II: Real Datasets (Strong Scaling Configurations)

ID	Matrix Name	Rows (M)	NNZ	Avg. NNZ	Dev. Std.	Structural Domain
1420	ASIC_680ks	682,712	1,693,767	2.48	4.16	Circuit (Sparse / Irregular)
1297	boyd2	466,316	1,500,397	3.22	194.91	Convex Opt. (Severe Spikes)
1881	Rucci1	1,977,885	7,791,168	3.94	0.34	Math Matrix (Rectangular)
2379	webbase-1M	1,000,005	3,105,536	3.11	25.35	Web Graph (Power-Law Hubs)
1419	ASIC_320ks	321,671	1,316,085	4.09	3.80	Circuit (Medium Regular)
1513	patents_main	240,547	560,943	2.33	1.94	Citation Graph (Ultra-Light)
39	bcsstk17	10,974	428,650	39.06	8.40	Stiffness Matrix (High Density)
1269	m_t1	185,000	9,753,570	52.72	1.18	Least Squares (Uniform Heavy)
38	bcsstk16	4,884	290,378	59.45	19.23	Stiffness Matrix (Small Dense)
1416	ASIC_100ks	99,190	578,890	5.84	13.91	Circuit (Small Regular Base)

2) Weak Scaling Dataset (Synthetic Generation): To properly evaluate Weak Scaling, the problem size must grow proportionally to the number of computational resources (N, $2N$, $4N$). We developed a custom Python generator creating two distinct synthetic topologies with a constant $D_{avg} \approx 3.0$:

- **ASIC-like (`synth_asic`):** Highly sparse but structurally uniform. Ensures perfect 1D cyclic load balancing while maintaining randomized non-zero column placements to simulate cache-miss irregularities.
- **BOYD2-like (`synth_boyd2`):** Designed for extreme load imbalance. Exactly 2% of the rows absorb 75% of the total Non-Zeros, mimicking power-law hub topologies and severely punishing 1D distribution schemes.

TABLE III: Synthetic Datasets (Weak Scaling Configurations)

Target GPUs	Matrix Name	Rows (M)	Total NNZ	Avg. D	Structural Profile
1 GPU	synth_asic_1gpu	200,000	$\approx$ 600,000	3.0	Balanced / Uniform
1 GPU	synth_boyd2_1gpu	200,000	600,000	3.0	Imbalanced / Hubs
2 GPUs	synth_asic_2gpu	400,000	$\approx$ 1,200,000	3.0	Balanced / Uniform
2 GPUs	synth_boyd2_2gpu	400,000	1,200,000	3.0	Imbalanced / Hubs
4 GPUs	synth_asic_4gpu	800,000	$\approx$ 2,400,000	3.0	Balanced / Uniform
4 GPUs	synth_boyd2_4gpu	800,000	2,400,000	3.0	Imbalanced / Hubs

IV. Performance Evaluation

Execution statistics were gathered across 1, 2, and 4 GPU configurations. Custom execution frameworks (Distributed CSR, COO, CSR-Vector, and cuSPARSE wrappers) were benchmarked against the multi-GPU SpMV baseline.

A. Execution Time and Computational Throughput

As depicted in Figure 1 and Figure 2, the vendor-optimized `cuSparse` framework retains dominance in raw execution speed and GFLOPS per SpMV, particularly on large matrices such as `Rucci1` and `webbase-1M`. However, among the custom implementations, the `CSR-Vector` kernel exhibits an impressive computational throughput (reaching peaks near 300 GFLOPS locally), effectively managing the warp-divergence typically associated with standard CSR implementations.

Fig. 1: Execution Time (s) per SpMV on Real Matrices (Log Scale).

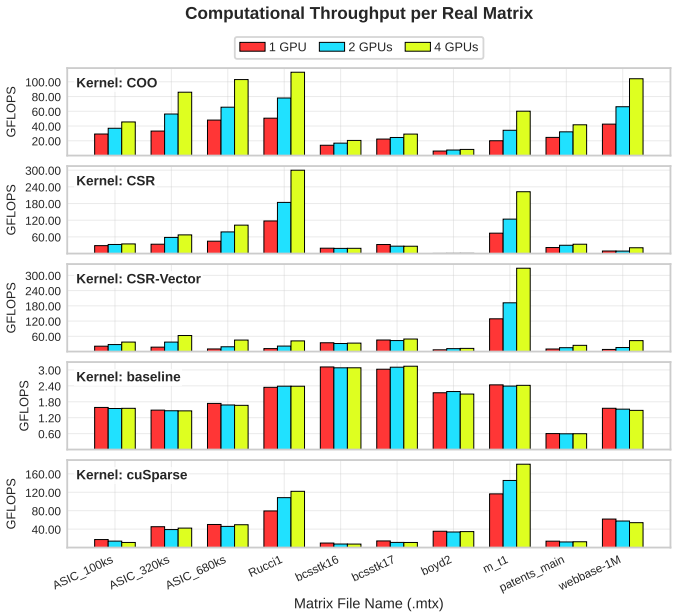

Fig. 2: Computational Throughput (GFLOPS) per Real Matrix.

B. Strong Scaling: Speedup and Efficiency

As visualized across the real datasets in Figures 3 and 4, the Strong Scaling capabilities display clear behavioral trends among the different configurations. While adding additional GPUs introduces non-trivial network overhead, the `CSR-Vector` approach scales the best among custom solutions, yielding an average speedup of $2.75\times$ on 4 GPUs with an efficiency of roughly 68.9%. Conversely, the `baseline` implementation completely fails to scale on our setup ($0.98\times$ on 4 GPUs with an efficiency drop down to 24%), largely due to the severe unoptimized `MPI_Bcast` overheads which our Ghost-Exchange implementation actively bypasses.
The individual matrix speedup and efficiency curves are detailed in Figures 3 and 4.

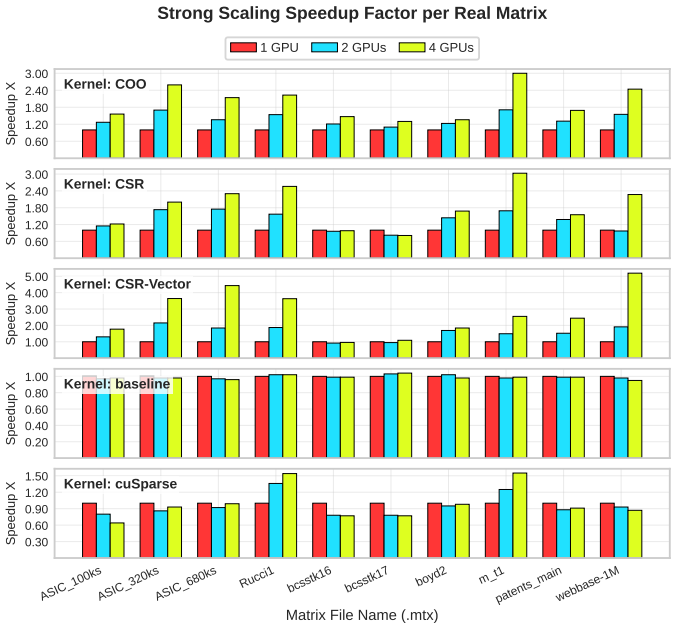

Fig. 3: Strong Scaling Speedup Factor per Real Matrix.

C. Communication vs Computation Breakdown

A crucial aspect of distributed GPU programming is understanding the cost of network synchronization. Figure 5 isolates the structural impact of MPI interactions, revealing the massive relative cost of the `MPI` Ghost Exchange. Because the local SpMV CUDA kernels execute extremely fast (often in the order of tens of microseconds), the time required to pack communication buffers, marshal data over the PCIe/NVLink interconnect, and wait for peer-to-peer non-blocking completion effectively dwarfs the pure computation time.
An analytical breakdown of the data shows that on 4 GPUs, the custom custom-tailored implementations are completely communication-bound: for instance, the `COO` kernel suffers from a Ghost Exchange overhead that is approximately $48\times$ higher than its pure compute time. Similarly, `CSR` and `CSR-Vector` register communication overhead indicators that are respectively $35\times$ and $39\times$ greater than the localized execution cost. Even the vendor-optimized `cuSparse`

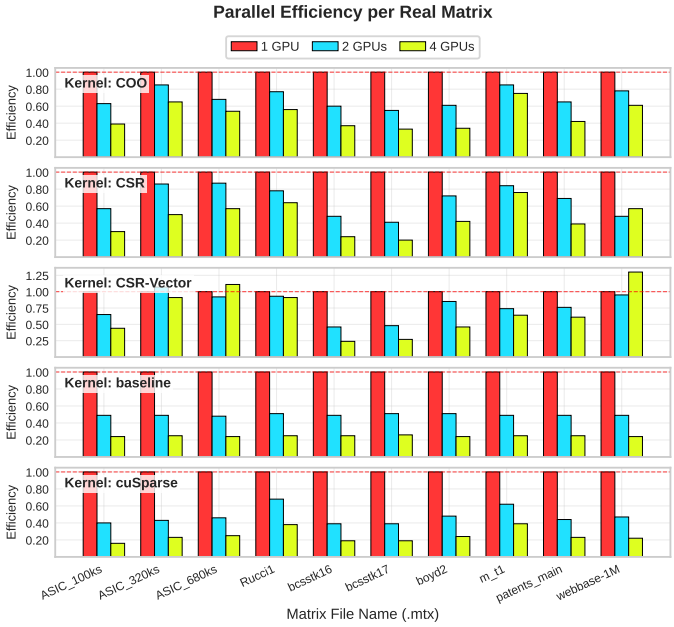

Fig. 4: Parallel Efficiency per Real Matrix (1, 2, and 4 GPUs).

pipeline is heavily penalized, with network-related wait times scaling up to $18\times$ higher than the pure numerical computation phase. This behavior confirms that increasing the GPU count beyond a certain threshold yields diminishing parallel returns, as the runtime becomes strictly limited by interconnect synchronization latencies. To explicitly quantify load balancing and network patterns across different architectures, the minimum, average, and maximum NNZ metrics per rank, alongside the discrete communication volume per rank, were systematically profiled, showing a direct match with the structural deviation metrics (σ) reported across the dataset collection.

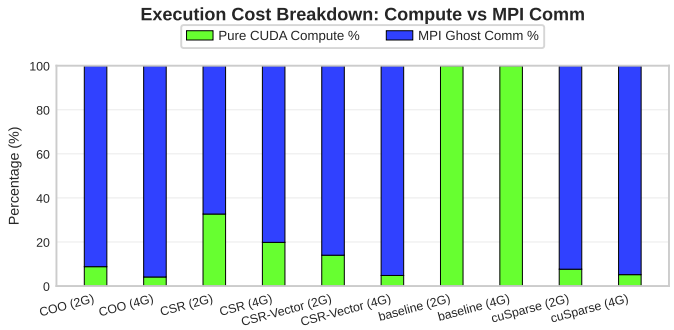

Fig. 5: Execution Cost Breakdown: Pure CUDA Compute vs. MPI Ghost Communication Overhead.

D. Weak Scaling Profiling

To isolate the structural impact on multi-node balancing, Weak Scaling was evaluated using synthetic datasets (Fig. 6). The custom implementations maintain excellent scaling efficiency on the `synth_asic` family (where 1D cyclic distribution ensures equal load), dropping only slightly below 0.90 efficiency on 4 GPUs. Conversely, the heavily imbalanced `synth_boyd2` datasets induce severe efficiency collapses (≈ 0.25 on 4 GPUs), as specific ranks are flooded by dense rows while peer ranks remain idle.

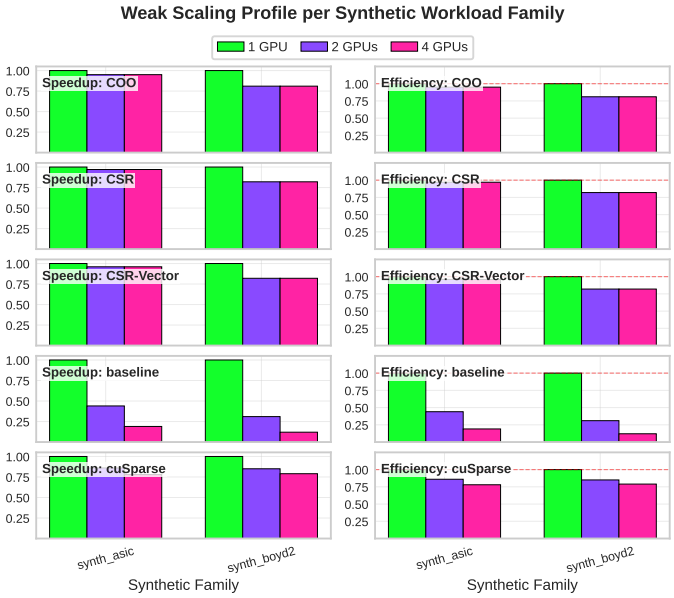

Fig. 6: Weak Scaling Efficiency and Speedup on Synthetic Workloads.

V. Discussion

A. When 1D Partitioning Works Well

1D cyclic partitioning delivers excellent execution scaling when applied to matrices featuring low row-density standard deviations ($\sigma \leq 4.16$), such as `Rucci1`, `m_t1`, and the synthetic `synth_asic` families. Because rows are distributed in an interleaved manner, localized variations in non-zero density are filtered out. This ensures that each independent GPU node receives an identical computational load.

B. When 2D Partitioning is Advantageous

1D cyclic partitioning fails when processing highly skewed power-law graphs or extreme optimization structures like `boyd2` ($\sigma = 194.91$) and our `synth_boyd2`. These datasets contain hyper-dense rows that act as hubs. Because a 1D allocation scheme assigns an entire row indivisibly to a single executor, the GPU that receives the hyper-dense row becomes a critical bottleneck. In this scenario, a 2D partitioning strategy is highly advantageous to break up highly connected hubs across multiple executors. As structurally argued in literature, advanced two-dimensional graph layouts and edge partitioning provide significant scaling improvements over 1D strategies when mapping scale-free networks onto thousands of cores [1].

C. When the Implementation is Bound by the Interconnect

As shown in the cost breakdown (Fig. 5), the implementation becomes strictly bound by the interconnect network when scaling to 4 GPUs. The time required by `MPI_Isend` and

`MPI_Irecv` routines to exchange non-local elements over the PCIe/network interface easily exceeds the pure execution time of the CUDA sparse multiplication kernel. Under these conditions, scaling to 4 GPUs yields decreasing performance returns, as the communication overhead dominates the total Time To Solution.

D. Largest Manageable Matrix Size

With the current memory layout, the maximum manageable matrix size is bounded primarily by the available memory on a single GPU node (NVIDIA A30 24GB). By replacing the global vector broadcast mechanism with a localized Ghost Entry exchange, this implementation avoids duplicating the full multi-million-element multiplier vector x on every device, allowing the cluster to easily handle large datasets exceeding 50 million non-zero elements per node context.

VI. Conclusion

This study expanded custom sparse matrix execution paths into a highly optimized distributed multi-GPU paradigm. Implementing a 1D cyclic distribution pattern effectively minimizes load imbalances for uniform structural topologies. Transitioning to a sparse peer-to-peer point-to-point network model built on GPU-Aware MPI concepts significantly reduces communication overheads compared to standard collective broadcasts. However, experimental results underscore the critical importance of matching data topology with appropriate parallel distribution models, as highly skewed matrices remain severely limited by interconnect performance and load imbalance under 1D constraints.

References

[1] E. G. Boman, K. D. Devine, and S. Rajamanickam, "Scalable Matrix Computations on Large Scale-Free Graphs Using 2D Graph Partitioning," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC13)*, 2013.

[2] J. Gao, W. Ji, and Y. Wang, "Optimization of Large-Scale Sparse Matrix-Vector Multiplication on Multi-GPU Systems," *ACM Transactions on Architecture and Code Optimization (TACO)*, vol. 21, no. 4, art. 69, 2024.